

FECAP

EmpathTech

Sistemas Operacionais e Computação em Nuvem

**São Paulo,
2026**

Integrantes

Carlos Roberto Santos Latorre - 24026336

Felipe Oluwaseun Santos Ojo - 24026245

Felipe Wakasa Klabunde - 24026544

Stephany Aliyah Guimarães Eurípedes de Paula - 24026434

SUMÁRIO

1.Introdução.....	1
2. Monitoramento e Algoritmos.....	4
3. Iniciando no COLAB.....	4
3.1 Conectando por meio das variáveis de ambiente:.....	4
3.2 Criando Tabela para armazenar informações referentes aos dados:.....	5
3.3 Coletando e Salvando os Dados:.....	5
3.4 Buscando dados:.....	7
3.5 Implementando algoritmos para análises:.....	7
3.5.1 Regressão Linear.....	7
3.5.2 K-Means.....	8
3.5.3 Árvore de Decisão.....	9
4.Conclusão.....	1

1.Introdução

Este arquivo tem como objetivo apresentar as métricas de monitoramento da instância em nuvem AWS RDS, por meio de 3 algoritmos aprendidos durante as aulas de Inteligência Artificial e Aprendizado de Máquina. Por meio deles, será possível monitorar o banco de dados e obter relatórios para a análise, considerando CPU, memória, disco e quantidade de processos.

2. Monitoramento e Algoritmos

Neste pequeno projeto de monitoramento de uma instância em nuvem, vamos utilizar os seguintes algoritmos: Regressão Linear, K-Means e Árvore de Decisão. Visamos aplicar o monitoramento no núcleo do projeto - o banco de dados. Dado que o MVP inicial funcionará localmente, devido às várias requisições que acessam API para entregar os frames de cada coleta, hospedar o projeto completamente em uma provedora de serviços em nuvem, pode se tornar algo muito caro logo, por se tratar da única instância em nuvem que utilizaremos, garantir o seu funcionamento correto se torna fundamental para garantir a persistência dos dados a cada evento realizado.

3. Iniciando no COLAB

3.1 Conectando por meio das variáveis de ambiente:

Como primeiro passo, devemos estruturar o código para que ele consiga acessar ao banco de dados e realizar as ações necessárias:

```
# =====  
# 2. CONEXÃO COM O BANCO  
# =====  
  
def conectar_banco():  
    return psycopg2.connect(  
        user=os.getenv("DB_USER"),  
        host=os.getenv("DB_HOST"),  
        database=os.getenv("DB_NAME"),  
        password=os.getenv("DB_PASSWORD"),  
        port=os.getenv("DB_PORT"),  
        sslmode="require"  
    )
```

3.2 Criando Tabela para armazenar informações referentes aos dados:

```
# =====  
# 3. NOVA TABELA PARA INSERÇÃO DE DADOS  
# =====  
  
def criar_tabela_monitoramento():  
    conn = conectar_banco()  
    cursor = conn.cursor()  
  
    cursor.execute("""  
        CREATE TABLE IF NOT EXISTS monitoramento_recursos (  
            id SERIAL PRIMARY KEY,  
            data_hora TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
            cpu_percent NUMERIC,  
            memoria_percent NUMERIC,  
            disco_percent NUMERIC,  
            processos INTEGER  
        );  
    """)  
  
    conn.commit()  
    cursor.close()  
    conn.close()  
  
    print("Tabela monitoramento_recursos pronta.")
```

3.3 Coletando e Salvando os Dados:

```
# =====
# 4. COLETAR E SALVAR DADOS
# =====

def coletar_e_salvar_metricas(qtd_amostras=30, intervalo=2):
    conn = conectar_banco()
    cursor = conn.cursor()

    print("Coletando métricas da instância...")

    for i in range(qtd_amostras):
        cpu = psutil.cpu_percent(interval=1)
        memoria = psutil.virtual_memory().percent
        disco = psutil.disk_usage("/").percent
        processos = len(psutil.pids())

        cursor.execute("""
            INSERT INTO monitoramento_recursos
            (cpu_percent, memoria_percent, disco_percent, processos)
            VALUES (%s, %s, %s, %s);
        """, (cpu, memoria, disco, processos))

        conn.commit()

        print(f"Amostra {i + 1}/{qtd_amostras} salva | CPU: {cpu}% | Memória: {memoria}% | Disco: {disco}%")

        time.sleep(intervalo)

    cursor.close()
    conn.close()

    print("Coleta finalizada.")
```

3.4 Buscando dados:

```
# =====  
# 5. BUSCAR DADOS DO BANCO  
# =====  
  
def buscar_dados():  
    conn = conectar_banco()  
  
    query = """  
        SELECT  
            id,  
            data_hora,  
            cpu_percent AS cpu,  
            memoria_percent AS memoria,  
            disco_percent AS disco,  
            processos  
        FROM monitoramento_recursos  
        ORDER BY id ASC;  
    """  
  
    df = pd.read_sql_query(query, conn)  
    conn.close()  
  
    df["cpu"] = pd.to_numeric(df["cpu"])  
    df["memoria"] = pd.to_numeric(df["memoria"])  
    df["disco"] = pd.to_numeric(df["disco"])  
    df["processos"] = pd.to_numeric(df["processos"])  
    df["tempo_seq"] = np.arange(len(df))  
  
    return df
```

3.5 Implementando algoritmos para análises:

3.5.1 Regressão Linear

A escolha deste algoritmo se deu pela possibilidade de prevermos o comportamento da CPU durante os eventos de coletas. A partir dele, conseguimos verificar suas variações a

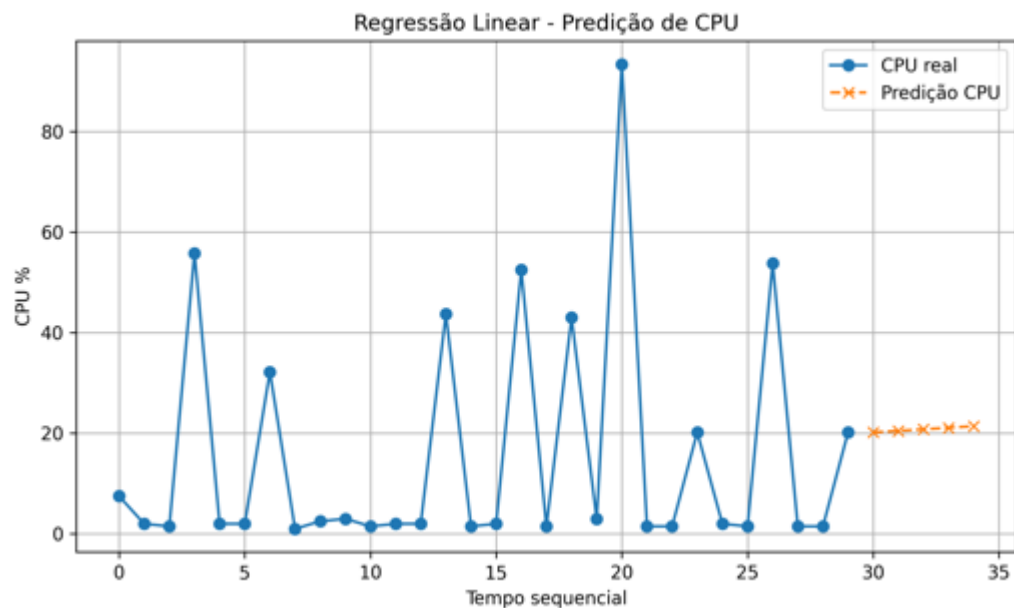
cada inserção de dados.

```
def aplicar_regressao_linear(df):
    X = df[["tempo_seq"]]
    y = df["cpu"]

    modelo = LinearRegression()
    modelo.fit(X, y)

    proximos_tempos = np.array([len(df), len(df) + 1, len(df) + 2, len(df) + 3, len(df) + 4])
    predicoes = modelo.predict(proximos_tempos)

    return predicoes
```



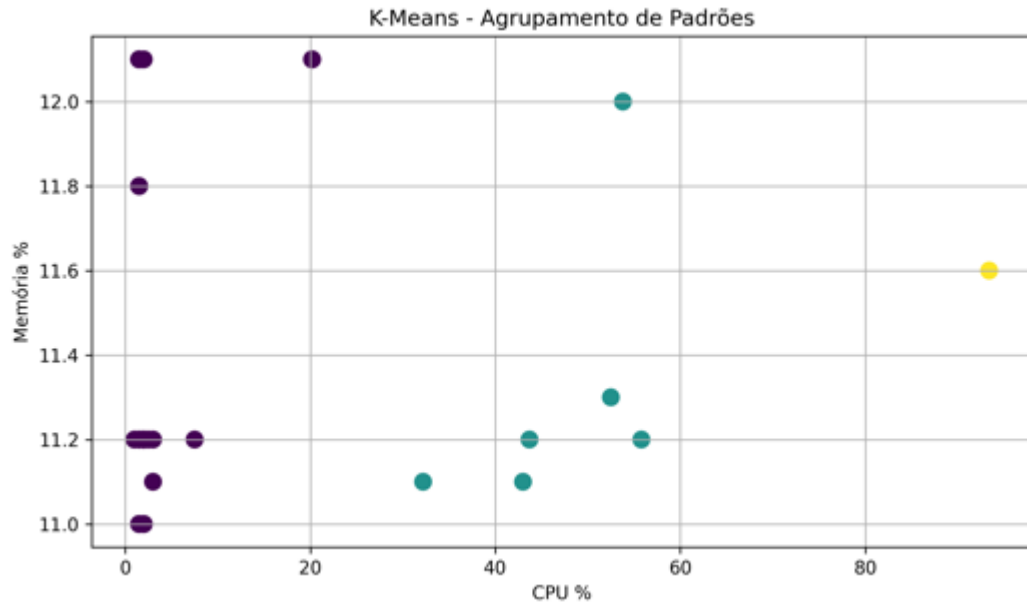
3.5.2 K-Means

Com o uso deste algoritmo conseguimos identificar os padrões de consumo dos recursos fornecidos, extinguindo a necessidade de definir manualmente cada situação.

```
def aplicar_kmeans(df):
    X = df[["cpu", "memoria", "disco"]]

    modelo = KMeans(n_clusters=3, random_state=42, n_init=10)
    clusters = modelo.fit_predict(X)

    return clusters
```

3.5.3 Árvore de Decisão

Com esse algoritmo, conseguimos detectar anomalias quando algum recurso passa do limite. O modelo aprende com base nos registros e movimentações para informar situações de atenção.

```
def aplicar_arvore_decisao(df):
    df = df.copy()

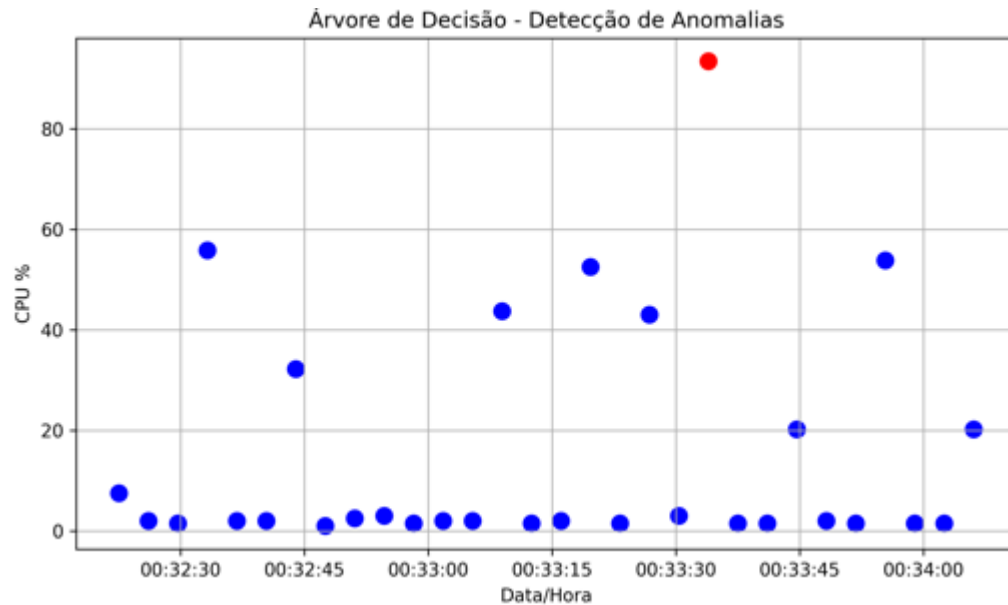
    df["anomalia"] = (
        (df["cpu"] > 75) |
        (df["memoria"] > 80) |
        (df["disco"] > 90)
    ).astype(int)

    X = df[["cpu", "memoria", "disco", "processos"]]
    y = df["anomalia"]

    modelo = DecisionTreeClassifier(max_depth=4, random_state=42)
    modelo.fit(X, y)

    predicoes = modelo.predict(X)

    return predicoes
```



Como podemos ver nos exemplos acima, cada algoritmo apresentou momentos de adversidades quando o banco de dados sofreu algum tipo de movimentação, esse tipo de informação é valiosa para podermos atender as necessidades e saber provisionar os recursos da maneira ideal, permitindo um bom manejo dos gastos e administração do sistema.

4. Conclusão

Uma vez que o banco de dados faz parte essencial da solução, ele atua como o local responsável por persistir todas as informações relevantes do sistema, garantindo que os dados coletados em cada evento sejam processados e analisados e permaneçam armazenados de forma segura e organizada. Dessa forma, é possível manter um histórico das métricas monitoradas, consultar registros anteriores e utilizar essas informações como base para a geração de gráficos, relatórios e análises de desempenho com os algoritmos de IA aplicados. Auxiliando a equipe de gestão na hora de provisionar os recursos e escalar conforme a necessidade do coletivo Lideranças Empáticas.